EXPERIMENTS 1-10

Name: Naresh M

Reg no: 192424381

Course code: DSA0503

Subject : Query Processing for Data Science

EXPERIMENT – 1

Aim

To write a Pandas program to select the distinct department IDs from the employees file.

Procedure

Step 1: Import the Pandas library.

Step 2: Read the employee dataset (employees.csv) using pd.read_csv().

Step 3: Store the dataset in a DataFrame.

Step 4: Select the **DEPARTMENT_ID** column.

Step 5: Use the unique() function to obtain all distinct department IDs.

Step 6: Display the distinct department IDs.

CODE

```
import pandas as pd
df = pd.read_csv("employees.csv")
print("Employee Dataset")
print(df)
distinct_departments = df["DEPARTMENT_ID"].unique()
print("\nDistinct Department IDs")
print(distinct_departments)
```

OUTPUT

```
========== RESTART: C:\Users\nares\Videos\dsa0503\experiments\exp 1.py =========
Employee Department Data:
    DEPARTMENT_ID       DEPARTMENT_NAME  MANAGER_ID  LOCATION_ID
0              10        Administration         200         1700
1              20             Marketing         201         1800
2              30            Purchasing         114         1700
3              40       Human Resources         203         2400
4              50              Shipping         121         1500
5              60                    IT         103         1400
6              70      Public Relations         204         2700
7              80                 Sales         145         2500
8              90             Executive         100         1700
9             100               Finance         108         1700
10            110            Accounting         205         1700
11            120              Treasury           0         1700
12            130         Corporate Tax           0         1700
13            140    Control And Credit           0         1700
14            150  Shareholder Services           0         1700
15            160              Benefits           0         1700
16            170         Manufacturing           0         1700
17            180          Construction           0         1700
18            190           Contracting           0         1700
19            200            Operations           0         1700
20            210            IT Support           0         1700
21            220                   NOC           0         1700
22            230           IT Helpdesk           0         1700
23            240      Government Sales           0         1700
24            250          Retail Sales           0         1700
25            260             Recruiting           0         1700
26            270                Payroll           0         1700

Distinct Department IDs:
[ 10  20  30  40  50  60  70  80  90 100 110 120 130 140 150 160 170 180
 190 200 210 220 230 240 250 260 270]
```

Result

Thus, the Pandas program to select the distinct department IDs from the employee dataset was executed successfully, and the distinct department IDs were displayed.

EXPERIMENT – 2

Aim

To write a Pandas program to display the IDs of employees who have done two or more jobs in the past.

Procedure

Step 1: Import the Pandas library.

Step 2: Read the employee job history dataset (job_history.csv) using pd.read_csv().

Step 3: Store the dataset in a DataFrame.

Step 4: Group the data based on **EMPLOYEE_ID**.

Step 5: Count the number of jobs for each employee.

Step 6: Filter employees whose job count is greater than or equal to 2.

Step 7: Display the Employee IDs.

Code:

```
import pandas as pd

df = pd.read_csv("job_history.csv")

print("Job History Dataset")

print(df)

job_count = df.groupby("EMPLOYEE_ID").size()
```

employees = job_count[job_count >= 2]

print("\nEmployees Who Have Done Two or More Jobs:")

print(employees.index.tolist())

OUTPUT:

```
Job History Dataset
   EMPLOYEE_ID  START_DATE    END_DATE      JOB_ID  DEPARTMENT_ID
0          102  13-01-2001  24-07-2006     IT_PROG             60
1          101  21-09-1997  27-10-2001  AC_ACCOUNT            110
2          101  28-10-2001  15-03-2005      AC_MGR            110
3          201  17-02-2004  19-12-2007      MK_REP             20
4          114  24-03-2006  31-12-2007    ST_CLERK             50
5          122  01-01-2007  31-12-2007    ST_CLERK             50
6          200  17-09-1995  17-06-2001     AD_ASST             90
7          176  24-03-2006  31-12-2006      SA_REP             80
8          176  01-01-2007  31-12-2007      SA_MAN             80
9          200  01-07-2002  31-12-2006  AC_ACCOUNT             90

Employees Who Have Done Two or More Jobs:
[101, 176, 200]
```

Result

Thus, the Pandas program to display the IDs of employees who have done **two or more jobs in the past** was executed successfully, and the employee IDs satisfying the condition were displayed.

EXPERIMENT – 3

Aim

To write a Pandas program to display the details of jobs in descending order based on the **JOB_TITLE** column.

Procedure

Step 1: Import the Pandas library.

Step 2: Read the jobs dataset (jobs.csv) using pd.read_csv().

Step 3: Store the data in a DataFrame.

Step 4: Sort the DataFrame by the **JOB_TITLE** column in descending order using sort_values().

Step 5: Display the sorted DataFrame.

CODE:

import pandas as pd

df = pd.read_csv("jobs.csv")

print("Original Job Details")

print(df)

sorted_jobs = df.sort_values(by="JOB_TITLE", ascending=False)

print("\nJob Details in Descending Order of Job Title")

print(sorted_jobs)

OUTPUT:

```
Job Details in Descending Order of Job Title
        JOB_ID                          JOB_TITLE  MIN_SALARY  MAX_SALARY
11      ST_MAN                      Stock Manager        5500        8500
12    ST_CLERK                        Stock Clerk        2008        5000
13    SH_CLERK                     Shipping Clerk        2500        5500
8       SA_REP                Sales Representative        6000       12008
7       SA_MAN                       Sales Manager       10000       20080
9       PU_MAN                  Purchasing Manager        8000       15000
10    PU_CLERK                    Purchasing Clerk        2500        5500
18      PR_REP  Public Relations Representative        4500       10500
6   AC_ACCOUNT                  Public Accountant        4200        9000
14      IT_PROG                          Programmer        4000       10000
0       AD_PRES                           President       20080       40000
16      MK_REP           Marketing Representative        4000        9000
15      MK_MAN                  Marketing Manager        9000       15000
17      HR_REP   Human Resources Representative        4000        9000
3       FI_MGR                    Finance Manager        8200       16000
1        AD_VP    Administration Vice President       15000       30000
2       AD_ASST         Administration Assistant        3000        6000
5       AC_MGR                Accounting Manager        8200       16000
4   FI_ACCOUNT                        Accountant        4200        9000
```

Result

Thus, the Pandas program to display the details of jobs in **descending order of the JOB_TITLE** was executed successfully, and the sorted job details were displayed.

EXPERIMENT – 4

Aim

To write a Pandas program to create a line plot of the historical stock prices of Alphabet Inc. between two specific dates.

Procedure

Step 1: Import the Pandas and Matplotlib libraries.

Step 2: Read the stock dataset using pd.read_csv().

Step 3: Convert the **Date** column into datetime format using dayfirst=True.

Step 4: Filter the records between **03-10-2016** and **07-10-2016**.

Step 5: Plot the **Close** price against the **Date**.

Step 6: Add title, labels, and grid.

Step 7: Display the graph.

CODE:

Import libraries

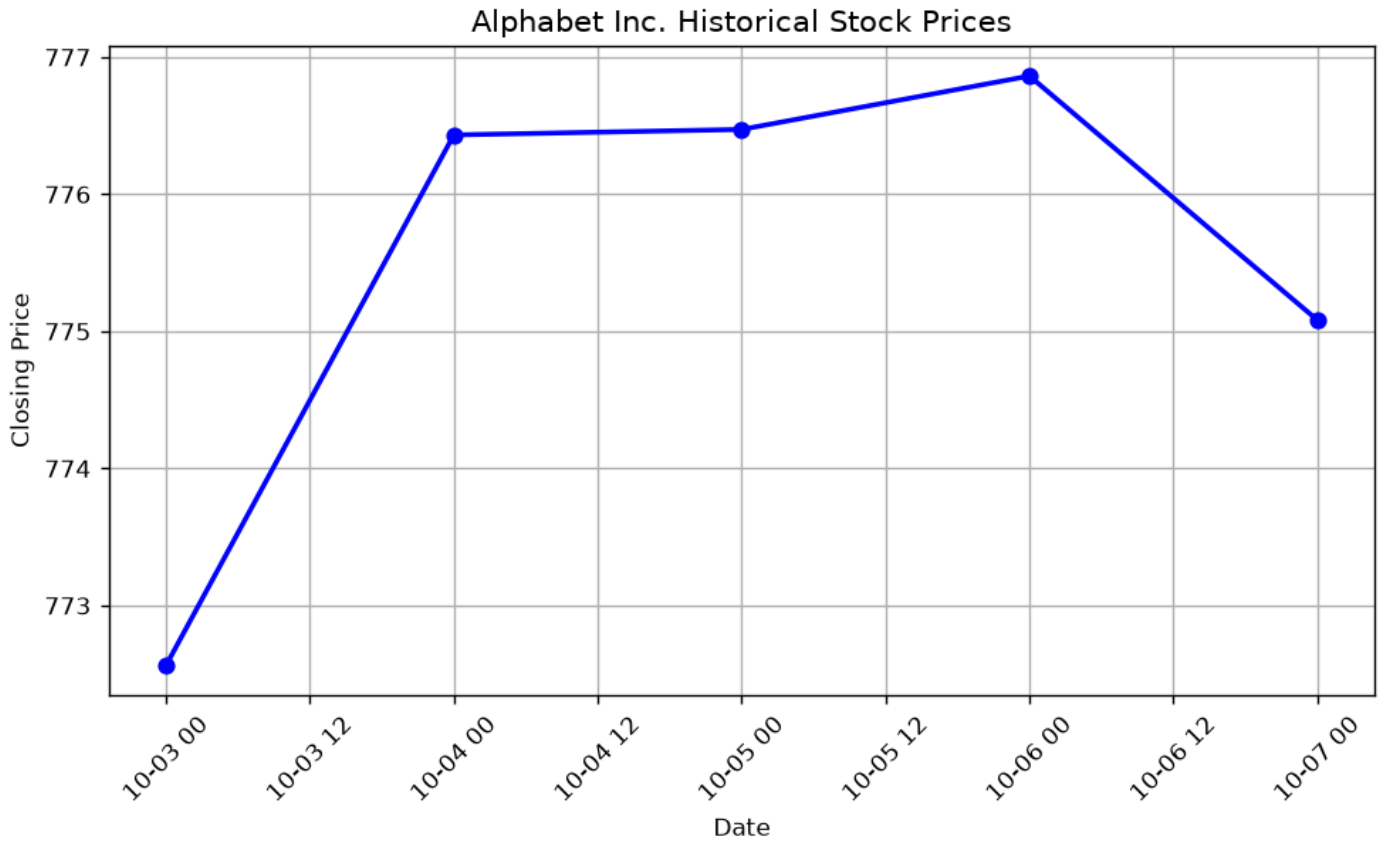
import pandas as pd

import matplotlib.pyplot as plt

df = pd.read_csv("alphabet_stock_data.csv")

```
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)
filtered_data = df[
    (df["Date"] >= "2016-10-03") &
    (df["Date"] <= "2016-10-07")
]
print(filtered_data)
plt.figure(figsize=(8,5))
plt.plot(filtered_data["Date"],
         filtered_data["Close"],
         marker='o',
         linewidth=2,
         color='blue')
plt.title("Alphabet Inc. Historical Stock Prices")
plt.xlabel("Date")
plt.ylabel("Closing Price")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```

OUTPUT:



Result

Thus, the Pandas program to create a **line plot of the historical stock prices of Alphabet Inc. between two specific dates** was executed successfully, and the required line graph was obtained.

EXPERIMENT – 5

Aim

To write a Pandas program to create a **bar plot** of the trading volume of Alphabet Inc. stock between two specific dates.

Procedure

Step 1: Import the Pandas and Matplotlib libraries.

Step 2: Read the stock dataset (alphabet_stock_data.csv).

Step 3: Convert the **Date** column into datetime format.

Step 4: Filter the data between **03-10-2016** and **07-10-2016**.

Step 5: Plot the **Volume** column using a bar chart.

Step 6: Add chart title, axis labels, and grid.

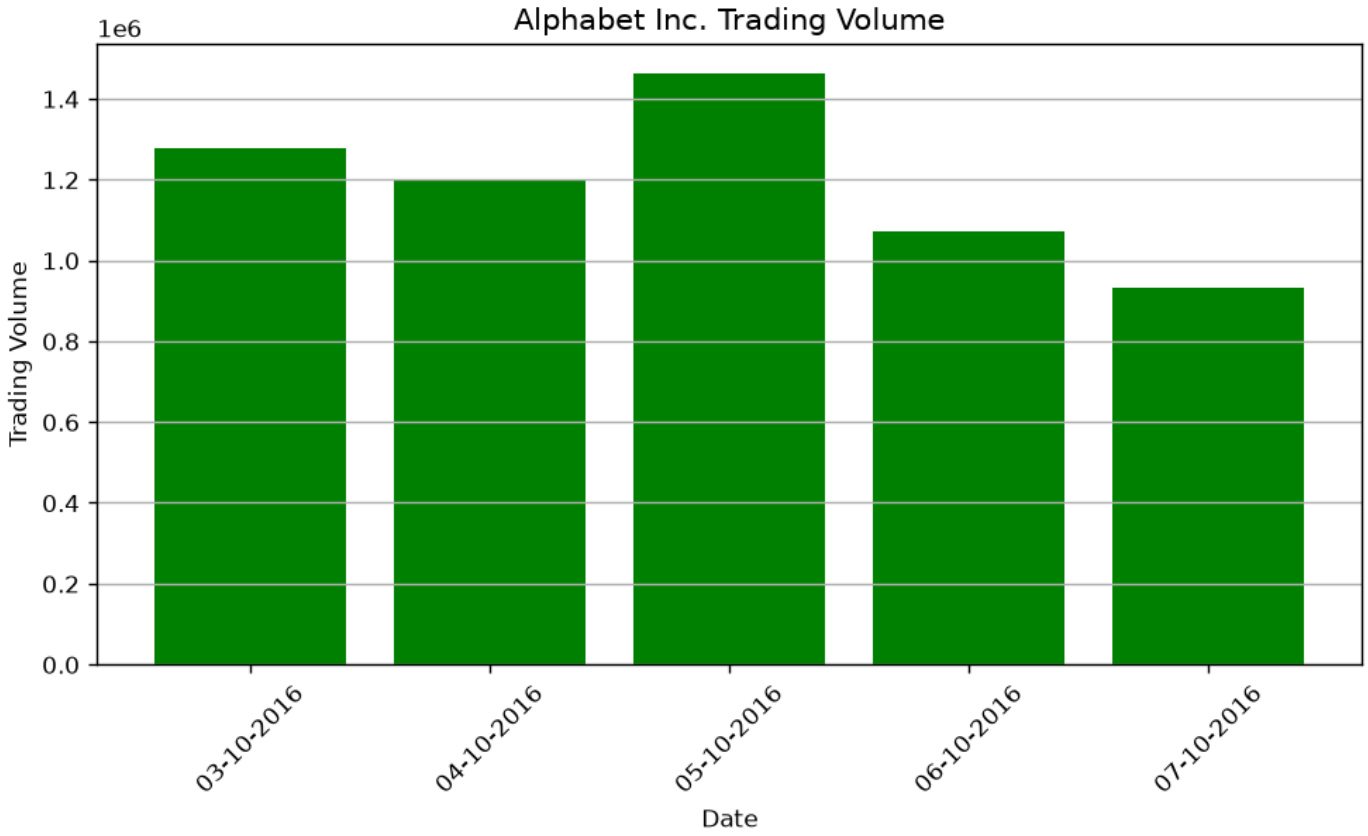
Step 7: Display the graph.

CODE:

import pandas as pd

```
import matplotlib.pyplot as plt
df = pd.read_csv("alphabet_stock_data.csv")
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)
filtered_data = df[
    (df["Date"] >= "2016-10-03") &
    (df["Date"] <= "2016-10-07")
]
print(filtered_data)
plt.figure(figsize=(8,5))
plt.bar(filtered_data["Date"].dt.strftime("%d-%m-%Y"),
        filtered_data["Volume"],
        color="green")
plt.title("Alphabet Inc. Trading Volume")
plt.xlabel("Date")
plt.ylabel("Trading Volume")
plt.xticks(rotation=45)
plt.grid(axis="y")
plt.tight_layout()
plt.show()
```

OUTPUT:



Result

Thus, the Pandas program to create a **bar plot of the trading volume of Alphabet Inc. stock between two specific dates** was executed successfully, and the required bar chart was displayed

EXPERIMENT – 6

Aim

To write a Pandas program to create a **scatter plot** of the trading volume and stock prices of Alphabet Inc. between two specific dates.

Procedure

Step 1: Import the Pandas and Matplotlib libraries.

Step 2: Read the stock dataset (alphabet_stock_data.csv).

Step 3: Convert the **Date** column into datetime format.

Step 4: Filter the records between **03-10-2016** and **07-10-2016**.

Step 5: Create a scatter plot with **Trading Volume** on the X-axis and **Closing Price** on the Y-axis.

Step 6: Add chart title, labels, and grid.

Step 7: Display the graph.

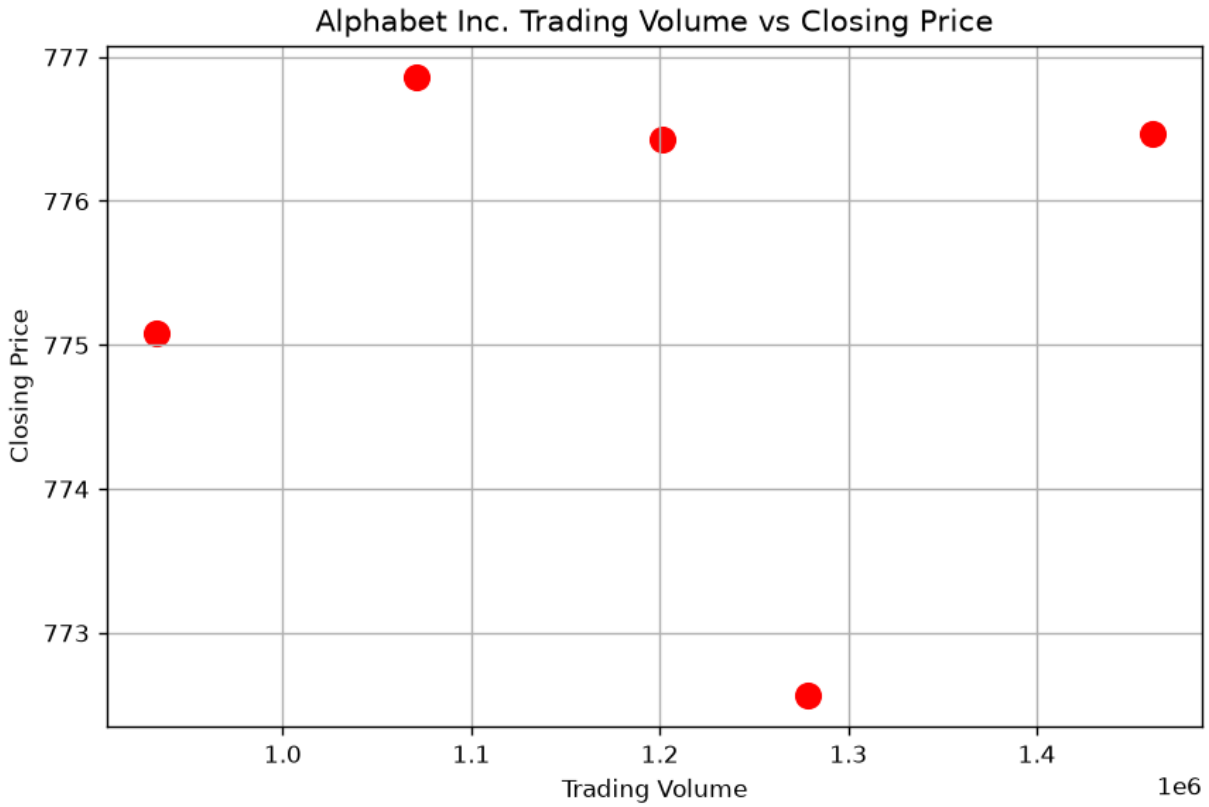
CODE:

```
import pandas as pd

import matplotlib.pyplot as plt
```

```
df = pd.read_csv("alphabet_stock_data.csv")
df["Date"] = pd.to_datetime(df["Date"], dayfirst=True)
filtered_data = df[
    (df["Date"] >= "2016-10-03") &
    (df["Date"] <= "2016-10-07")
]
print(filtered_data)
plt.figure(figsize=(8,5))
plt.scatter(filtered_data["Volume"],
            filtered_data["Close"],
            color="red",
            s=100)
plt.title("Alphabet Inc. Trading Volume vs Closing Price")
plt.xlabel("Trading Volume")
plt.ylabel("Closing Price")
plt.grid(True)
plt.show()
```

OUTPUT:



Result

Thus, the Pandas program to create a **scatter plot of the trading volume and closing stock prices of Alphabet Inc. between two specific dates** was executed successfully, and the required scatter plot was displayed.

EXPERIMENT – 7

Aim

To write a Pandas program to create a **Pivot Table** and find the **maximum** and **minimum sale value** of each item using the sales_data dataset.

Procedure

Step 1: Import the Pandas library.

Step 2: Read the sales_data.csv file using pd.read_csv().

Step 3: Store the data in a DataFrame.

Step 4: Create a Pivot Table using the **Item** column as the index.

Step 5: Select **Sale_amt** as the values column.

Step 6: Apply **maximum** and **minimum** aggregation functions.

Step 7: Display the Pivot Table.

CODE:

```
import pandas as pd
df = pd.read_csv("sales_data.csv")
print("Sales Dataset")
print(df)
pivot_table = pd.pivot_table(
    df,
    values="Sale_amt",
    index="Item",
    aggfunc=["max", "min"]
)
print("\nMaximum and Minimum Sale Value of Each Item")
print(pivot_table)
```

OUTPUT:

```
Maximum and Minimum Sale Value of Each Item
                 max      min
             Sale_amt Sale_amt
Item
Cell Phone      14400     6075
Desk              250      250
Home Theater    40500    14000
Television     113810    38336
Video Games       936      936
```

Result

Thus, the Pandas program to create a **Pivot Table** and find the **maximum** and **minimum sale value** of each item was executed successfully, and the required Pivot Table was displayed.

EXPERIMENT – 8

Aim

To write a Pandas program to create a **Pivot Table** and find the **item-wise units sold** using the sales_data dataset.

Procedure

Step 1: Import the Pandas library.

Step 2: Read the sales_data.csv file using pd.read_csv().

Step 3: Store the data in a DataFrame.

Step 4: Create a Pivot Table using the **Item** column as the index.

Step 5: Select the **Units** column as the values.

Step 6: Apply the **sum** aggregation function to calculate the total units sold for each item.

Step 7: Display the Pivot Table.

CODE:

```
import pandas as pd
df = pd.read_csv("sales_data.csv")
print("Sales Dataset")
print(df)
pivot_table = pd.pivot_table(
    df,
    values="Units",
    index="Item",
    aggfunc="sum"
)
print("\nItem-wise Units Sold")
```

print(pivot_table)

OUTPUT:

```
Item-wise Units Sold
              Units
Item
Cell Phone       91
Desk              2
Home Theater    308
Television      509
Video Games      16
```

Result

Thus, the Pandas program to create a **Pivot Table** and find the **item-wise units sold** was executed successfully, and the required Pivot Table was displayed.

EXPERIMENT – 9

Aim

To write a Pandas program to create a **Pivot Table** that displays the **total sale amount for each region** using the sales_data dataset.

Procedure

Step 1: Import the Pandas library.

Step 2: Read the sales_data.csv file using pd.read_csv().

Step 3: Store the data in a DataFrame.

Step 4: Create a Pivot Table using the **Region** column as the index.

Step 5: Select the **Sale_amt** column as the values.

Step 6: Apply the **sum** aggregation function to calculate the total sales for each region.

Step 7: Display the Pivot Table.

CODE:

```
import pandas as pd
df = pd.read_csv("sales_data.csv")
print("Sales Dataset")
print(df)
pivot_table = pd.pivot_table(
    df,
    values="Sale_amt",
    index="Region",
    aggfunc="sum"
```

)

print("\nRegion-wise Total Sale Amount")

print(pivot_table)

OUTPUT:

```
Region-wise Total Sale Amount
         Sale_amt
Region
Central    393943
East       286076
West       105424
```

Result

Thus, the Pandas program to create a **Pivot Table** and find the **region-wise total sale amount** was executed successfully, and the required Pivot Table was displayed.

EXPERIMENT – 10

Aim

To write a Pandas program to create a **Pivot Table** that displays the **total sale amount** for each **Region** and **Item** using the sales_data dataset.

Procedure

Step 1: Import the Pandas library.

Step 2: Read the sales_data.csv file using pd.read_csv().

Step 3: Store the data in a DataFrame.

Step 4: Create a Pivot Table using **Region** as the index and **Item** as the columns.

Step 5: Select the **Sale_amt** column as the values.

Step 6: Apply the **sum** aggregation function.

Step 7: Display the Pivot Table.

CODE:

```
import pandas as pd

df = pd.read_csv("sales_data.csv")

print("Sales Dataset")

print(df)

pivot_table = pd.pivot_table(
    df,
    values="Sale_amt",
    index="Region",
```

```
    columns="Item",
    aggfunc="sum",
    fill_value=0
)
print("\nRegion-wise and Item-wise Total Sale Amount")
print(pivot_table)
```

OUTPUT:

```
Region-wise and Item-wise Total Sale Amount
Item     Cell Phone  Desk  Home Theater  Television  Video Games
Region
Central        6075   250         39000      348618            0
East          14400     0        115000      155740          936
West              0     0             0      105424            0
```

Result

Thus, the Pandas program to create a **Pivot Table** showing the **region-wise and item-wise total sale amount** was executed successfully, and the required Pivot Table was displayed.